# TP 13 : Évaluer et séparer

*Ce TP est à réaliser en OCaml.*

## Préliminaires

On considère ici une variante du problème du voyageur de commerce où l'on cherche un chemin hamiltonien de poids minimal et non un cycle hamiltonien. On rappelle qu'un chemin hamiltonien est un chemin qui passe une et unique fois par chacun des sommets du graphe. Plus formellement, il s'agit du problème suivant :
Problème `Voyageur itinérant`

* Instance : un graphe complet et pondéré $G$ et un entier $k$.
* Question : le graphe $G$ admet-il un chemin qui passe par tous les sommets de poids total inférieur à $k$ ?

En pratique, on cherchera à trouver la valeur d'un chemin hamiltonien de longueur minimale.

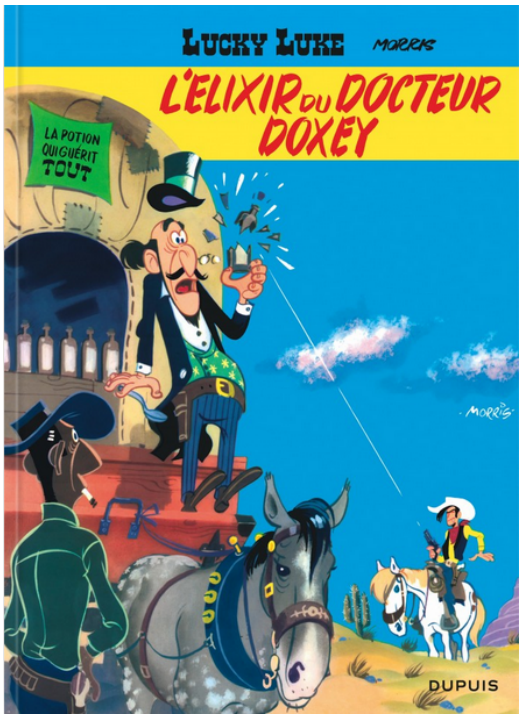


FIGURE 1 – Un exemple de voyageur itinérant

On représente en OCaml un graphe pondéré complet par une matrice d'adjacence pondérée.

```
type graphe = int array array
```

Ainsi, si g est un objet de type graphe, `g.(0).(1)` correspond au poids de l'arête entre les sommets 0 et 1.

## Recherche par énumération

Avant de réaliser un programme de séparation et division qui résout ce problème, il faut avoir une méthode d'énumération des solutions potentielles. Pour cela, on propose l'algorithme suivant qui étant donnée une permutation de $[\![0, n-1]\!]$, la modifie en une autre permutation :

Pour représenter une permutation, on utilisera ici un tableau où chaque entier de 0 à $n-1$ apparaît exactement une fois.

**Algorithme 1** Suivant($\sigma$)

$x \leftarrow$ plus long suffixe strictement décroissant de $\sigma$
$i \leftarrow$ l'indice du premier élément de $x$ dans $\sigma$
$j \leftarrow$ l'indice du plus petit entier $\sigma_k$ tel que $\sigma_k$ soit strictement supérieur à $\sigma_{i-1}$ et $k \geq i$
Échanger $\sigma_{i-1}$ et $\sigma_j$
Inverser les éléments de $\sigma$ à partir de la position $i$.

```
type permutation = int array
```

**Question 1 :** Écrire une fonction `plus_long_suffixe : permutation -> int` qui, étant donnée une permutation, renvoie la position $i$ comme indiquée dans l'algorithme.

**Question 2 :** Écrire une fonction `trouver_j : permutation -> int` qui, étant donnée une permutation, renvoie le $j$ correspondant.

**Question 3 :** Écrire une fonction `inverser : permutation -> int -> unit` qui, étant donnée une permutation et un indice $i$ de celle-ci, modifie celle-ci pour inverser l'ordre des éléments à partir de $i$.

**Question 4 :** Écrire une fonction `permutation_suivante : permutation -> unit` qui, étant donnée une permutation, la modifie selon l'algorithme proposé.

**Question 5 :** Il existe un cas pour lequel l'algorithme ne fonctionne pas, quel est ce cas ?

On admet que si l'on part de la permutation identité, l'application successive de l'algorithme énumère toutes les permutations jusqu'à atteindre ce cas.

**Question 6 :** Écrire une fonction `valeur_chemin : graphe -> permutation -> int` qui, étant donnée une permutation des sommets du graphe, renvoie le poids du chemin ainsi obtenu.

**Question 7 :** Écrire une fonction `trouver_min_enum : graphe -> permutation` qui, étant donné un graphe, renvoie le chemin hamiltonien de poids minimum.

**Question 8 :** Si le graphe possède $n$ sommets, quel est la complexité de la fonction précédente ?

## Séparation et évaluation

Pour appliquer la méthode de séparation et évaluation, il faut d'abord avoir un algorithme de retour sur trace.Pour cela, on se munit d'un tableau de booléens indiquant si l'on est déjà passé par les sommets. Lorsque l'on cherche à construire une solution, on teste en ajoutant chacun des sommets possibles au chemin que l'on a déjà construit. On ajoute aussi deux attributs, l'un calculant la longueur du chemin déjà construit et l'autre indiquant le sommet composant l'extrémité du chemin.

On représente finalement un chemin partiel par le type suivant :

```
type chemin_partiel =
{
    vus : bool array;
    longueur : int;
    extremite : int;
}
```

**Question 9 :** Écrire une fonction `retour_sur_trace : graphe -> chemin_partiel -> int` qui, étant donnés un graphe et un chemin représenté par son extrémité et le tableau des sommets déjà utilisés, renvoie la longueur du plus court chemin depuis l'extrémité ne passant exactement que par les sommets pas encore empruntés.

**Question 10 :** En déduire une fonction `trouver_min_retour : graphe -> int` qui, étant donné un graphe, renvoie la valeur du plus court chemin hamiltonien de ce graphe.

On se propose de borner la pertinence d'une solution partielle par la longueur du chemin partiel déjà construit. On admet qu'il s'agit d'une heuristique admissible.

**Question 11 :** Écrire une fonction `separer_evaluer : graphe -> chemin_partiel -> int` qui, étant donnés un graphe, un chemin partiel construit jusqu'à présent, renvoie la longueur du plus court chemin hamiltonien que l'on peut obtenir en poursuivant le chemin partiel. On ignorera les branches dont l'heuristique nous indique qu'elles ne sont pas pertinentes.

**Question 12 :** En déduire une fonction `trouver_min_rapide : graphe -> int` qui, étant donné un graphe, renvoie la longueur du plus court chemin hamiltonien de ce graphe en utilisant l'algorithme de séparation et évaluation.

## Construction d'un chemin

Jusqu'à présent, on récupérait seulement la longueur d'un plus court chemin. On cherche maintenant à récupérer le chemin qui minimise la longueur. Pour cela, on réutilise le type `permutation` de la première partie.

**Question 13 :** Écrire une fonction `trouver_chemin : graphe -> chemin` qui, étant donné un graphe, renvoie le plus chemin minimisant la longueur.